

AI PRODUCT PROTOTYPE CHALLENGE

Submission Report

Smart First-Aid Assistant

Candidate: Gayath Dahanayaka

Use case chosen: Basic first-aid and health guidance for the general public in Sri Lanka

1. Research Process & Findings

I picked a first-aid and basic health-guidance assistant for the use case. It felt like a genuinely useful scenario for Sri Lanka, and it naturally forced me to hit all three core requirements: avatar, AI conversation, and configurable knowledge, instead of building something that's just open-ended chat with a face on it.

3D avatar / character platform

- I originally built the avatar flow around Ready Player Me. It was free, well-documented, and I had it fully working. Then partway through development it just stopped loading, with no clear error.
- It turned out Ready Player Me had been acquired by Netflix, and its entire public avatar platform was shut down on January 31, 2026, with no public successor. It wasn't a bug in my code; the service I depended on had simply stopped existing.
- I switched to Avaturn instead, which offered the same free demo tier and the same "creator iframe to exported .glb URL" flow, so most of my code didn't need to change. The main lesson: a prototype that depends on a live third-party service can break without warning, and keeping avatar creation separate from avatar rendering made switching providers a same-day fix instead of a rewrite

Conversational AI / LLM platform

- Compared OpenAI's API, an on-device/open-source model, and Google's Gemini API. The challenge explicitly discourages paid subscriptions, so the decision came down to which free tier was workable: Gemini's free tier supports both chat completion and text embeddings, which is what made a self-built RAG pipeline realistic within the effort budget.
- Finding: Google periodically retires specific dated model snapshots (e.g. a fixed gemini-2.5-flash string) for newer API keys/projects faster than for older ones, which surfaced as a 404 "no longer available to new users" error during testing. Switching to the gemini-flash-latest alias resolved this permanently, since it always resolves to Google's current recommended model.
- Finding: the Gemini free tier caps embedding calls at roughly 1,000 requests/day. Re-embedding a multi-page PDF on every backend restart (which nodemon triggers on each file save) burns through that quota within a single afternoon of development — this directly motivated the disk-caching design decision described in Section 3.

Grounding approach (RAG vs. plain chat)

- A plain LLM call without grounding risks confidently-worded but incorrect first-aid advice, which is unacceptable for a health-guidance tool. Retrieval-Augmented Generation (RAG) over an official IFRC first-aid guideline PDF was chosen so every answer is anchored to a specific, citable source rather than the model's unverified general knowledge.

Language coverage

- Sri Lankan users commonly mix English, Sinhala and "Singlish" (romanised Sinhala) in everyday typing. The assistant was tested in all three and the system prompt instructs the model to reply in whichever of the three the user used, so the tool is usable without forcing a language switch during an emergency.

2. Design & Architecture

The system has three layers: a knowledge-preparation step that runs once, a retrieval-and-generation backend, and a React client that renders the avatar and chat.

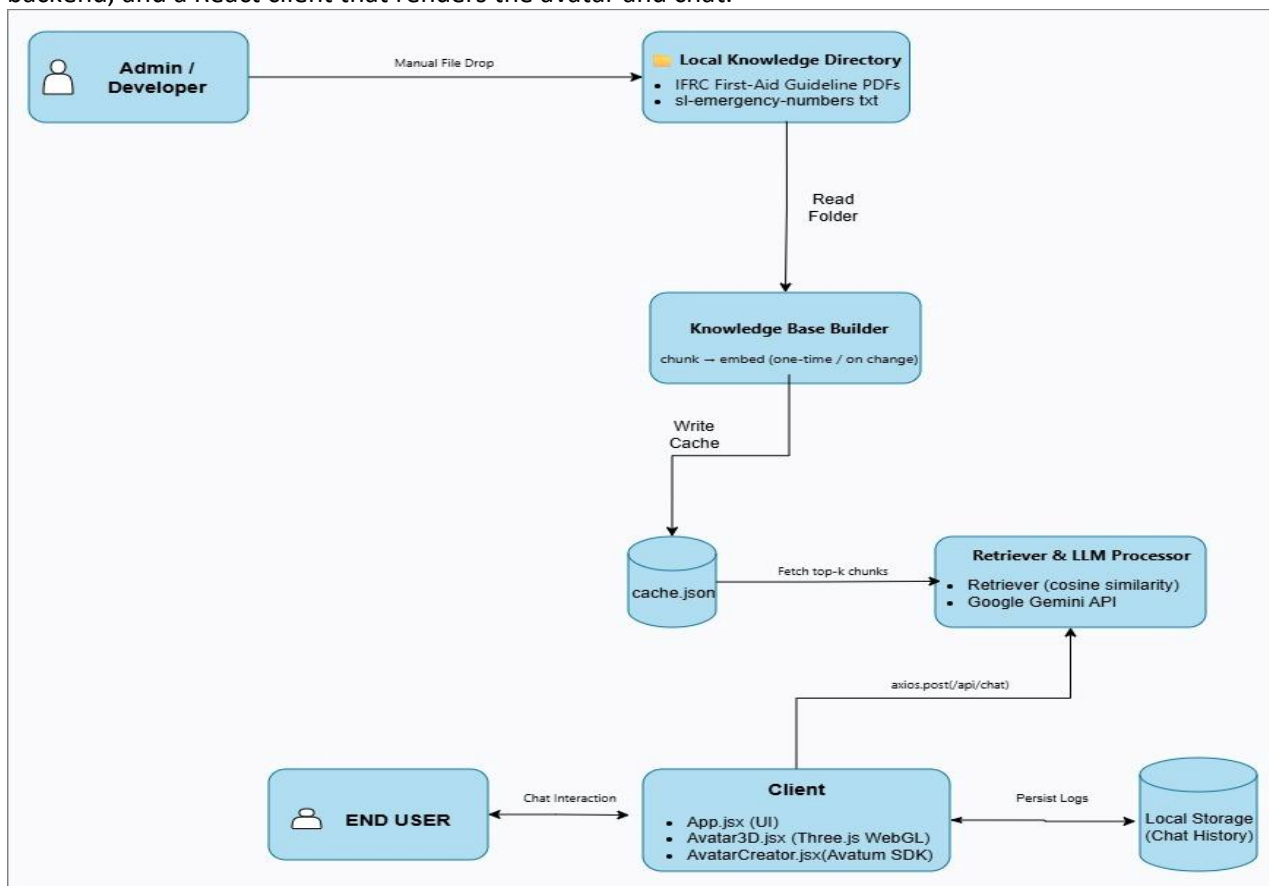


Diagram link - <https://drive.google.com/file/d/1rmEK8HVnpTX0Ea23KyGKo7JcEv7gxc3d/view?usp=sharing>

- AvatarCreator.jsx embeds the Avaturn SDK inside a container div. Once the user finishes customizing, it exports a hosted .glb URL back to the app through an onAvatarCreated callback.
- Avatar3D.jsx loads that .glb with useGLTF, normalizes it to a fixed height regardless of the source model's scale, and re-poses the arms out of the default T-pose using a rig-agnostic, self-correcting bone-alignment routine. It renders inside a react-three-fiber Canvas with the camera locked to a head-and-shoulders framing rather than a full-body shot.
- The same component captures a one-time PNG snapshot of the posed avatar (via canvas.toDataURL) and hands it up to App.jsx, which reuses it as the small chat-bubble icon, since mounting a full WebGL canvas per chat message would be wasteful.
- App.jsx owns chat state, persists it (plus the chosen avatar and facing direction) to localStorage behind try/catch guards so it doesn't crash in private-browsing mode, pings /api/health to show an honest "Ready / Reconnecting" status, and sends each message with prior history to /api/chat.
- On the backend, the knowledge-base builder chunks the IFRC guideline PDF, embeds each chunk with Gemini's embedding model, and writes the vectors to a disk cache, computed once rather than on every restart.
- /api/chat embeds the incoming question, retrieves the most relevant cached chunks by cosine similarity, builds a bilingual safety-first system prompt around them, and calls gemini-flash-latest for the reply, returning it to the client along with which sources it drew from.

3. Design Decisions

- **Disk-cached embeddings, not re-computed per run** - : The free-tier daily embedding quota (~1,000 requests) doesn't survive repeated dev-server restarts if every restart re-embeds the whole knowledge base. Embeddings are computed once and cached to a JSON file; the cache only rebuilds if the source documents actually change.
- **Batched embedding requests** -: Chunks are embedded 50-at-a-time per API call instead of one request per chunk, to stay well under the per-minute rate limit as well as the daily cap.
- **Model alias over a pinned version** -: Using gemini-flash-latest instead of a dated model string avoids the app silently breaking whenever Google retires a specific snapshot for new accounts, which happened once already during development.
- **Decoupling avatar creation from avatar rendering** -: AvatarCreator.jsx (the third-party integration) and Avatar3D.jsx (the rendering logic) are separate components. When Ready Player Me shut down, only the creator component needed replacing.
- **Rig-agnostic pose correction** -: Rather than hardcoding rotation angles for one specific avatar rig, the bone-fixing routine measures its own result and self-corrects direction if the first attempt moves a limb the wrong way, making it portable across avatar providers.
- **Head-and-shoulders portrait framing over full-body** -: A circular, face-focused camera crop reads as a professional assistant profile picture and fits a compact sidebar far better than a full-body shot.
- **Client-side persistence over a backend database** -: Chat history and avatar choice persist via localStorage rather than a server-side store, which keeps the scope appropriate for an 8-hour prototype while still giving users a non-frustrating experience across page refreshes.

- **Safety-first system prompt** -: Because the assistant gives health guidance, any life-threatening keywords trigger an instruction to call emergency services first, before any other content, treated as non-negotiable rather than a nice-to-have.

4. Implementation Process

I built this in dependency order: backend and knowledge base first, since the frontend has nothing to talk to without them.

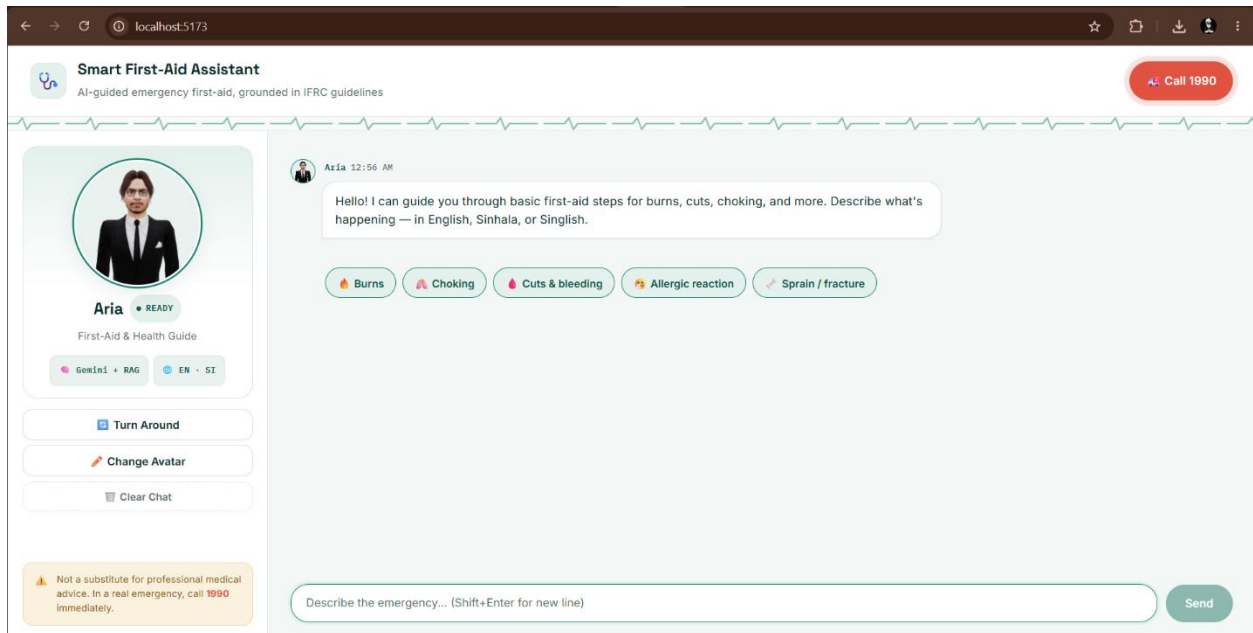
- Repo and backend scaffold: Express server, environment config, .gitignore.
- Knowledge base documents: sourced the IFRC first-aid guideline PDF and a Sri Lanka emergency-numbers reference file.
- RAG engine: PDF text extraction, chunking, batched embedding with retry/backoff, disk caching, cosine-similarity retrieval.
- Chat endpoint: wired retrieval into a bilingual, safety-first system prompt and Gemini's chat API.
- Frontend scaffold: React + Vite base app.
- Avatar integration: built against Ready Player Me, then rebuilt against Avaturn after RPM's shutdown (see Section 1); wrote the T-pose fix and portrait-framing camera logic.
- Chat UI: message list, input, source citations, localStorage persistence for chat and avatar state.
- UI polish: assistant identity card, live status indicator, response-latency badge, styled disclaimer, responsive layout.
- Testing: verified first-aid scenarios (burns, bleeding, choking, CPR, fractures, poisoning, fainting, snake bite) in English, Sinhala, and Singlish.

5. Technologies & Platforms Used

Category	Technology / Service	Notes
Frontend	React 18 + Vite	UI framework / build tool
3D rendering	Three.js, @react-three/fiber, @react-three/drei	Avatar rendering in-browser
Avatar platform	Avaturn SDK (@avaturn/sdk)	Free tier; replaced Ready Player Me after its Jan 2026 shutdown
Backend	Node.js + Express	REST API server
AI (chat)	Google Gemini API (@google/genai), gemini-flash-latest	Free tier; conversational responses
AI (embeddings)	Google Gemini API, gemini-embedding-001	Free tier; RAG retrieval
PDF processing	pdf-parse (v2)	Extracts text from the IFRC guideline PDF

Category	Technology / Service	Notes
HTTP client	axios	Frontend ↔ backend requests
Client persistence	Browser localStorage	Chat history, avatar choice/facing
Server persistence	Disk-based JSON cache	Pre-computed embeddings, avoids re-embedding on restart

6. Working Prototype



← → ↺ 📍 localhost:5173

☆ 📄 ⬇️ 👤 ⋮



Smart First-Aid Assistant

AI-guided emergency first-aid, grounded in IFRC guidelines

📞 Call 1990



Aria

• READY

First-Aid & Health Guide

Gemini • RAG

EN • SI

74425ms

Turn Around

Change Avatar

Clear Chat

⚠️ Not a substitute for professional medical advice. In a real emergency, call 1990 immediately.

Aria 12:58 AM

If the burn is large, deep, or if the person is showing signs of shock, please call emergency services immediately on 1990.

Here are the immediate first-aid steps:

1. **Cool the burn:** Keep the burned hand under clean, cool running water for at least 10 to 20 minutes. Do not use ice or ice-cold water, as this can worsen the injury.
2. **Remove jewellery/clothing:** Gently remove any rings, watches, or clothing near the burn, but only if they are not stuck to the skin.
3. **Dress the burn:** Cover the burn with a clean, non-stick dressing (such as hydrogel) to keep it moist. If a dressing is not available, you can apply vaseline or honey.
4. **Do not pop blisters:** If any blisters form, leave them intact to prevent infection.

Do you need more detail on this?

Disclaimer: This is not professional medical advice.

📄 who-first-aid-guidelines.pdf

Copy

Describe the emergency... (Shift+Enter for new line)

Send



Smart First-Aid Assistant

AI-guided emergency first-aid, grounded in IFRC guidelines

📞 Call 1990



Aria

• READY

First-Aid & Health Guide

Gemini • RAG

EN • SI

16147ms

Turn Around

Change Avatar

Clear Chat

⚠️ Not a substitute for professional medical advice. In a real emergency, call 1990 immediately.

Aria 01:03 AM

Sarpayek kavaama eya jeevitha tharthjanayak wenna puluwan, e nisa wahaama 1990 call karala hadisi vaidhya sahaya labaganna.

Mulaika prathamaadhara piyawara mevaayi:

1. **Nidi karavanna saha nishchala karanna:** Rogiyava pahasu asanayaka nidi karava, anga helavenna nodi nishchalawa thabanna (visa anga pura pathirena eka adu karanna).
2. **Aabarana saha thada adum galavanna:** Idimenna puluwan nisa, thuvama vuna limb eke (athe ho paaye) thiyena ringu, orlosu saha thada adum ikmanatama galavanna.
3. **Bandage ekak badinna:** Thuvamayuna limb eka non-elastic bandage ekakin nethnam thiyena adum kallekin bada nishchala karanna. Eya hadawathata wada pahathin thabanna.
4. **Thuvamaya sodanna:** Piriisidu jalayen thuvamaya sodanna. (Sarpaya allanna yanna epa, puluwan nam photoyak ganna).
5. **Karanna EPA deval:** Thuvamaya kapanna epa, visa uranna epa, ice thiyanna epa, madinna epa, saha tourniquet (thada lanu) badinna epa.

Oyata me pilibada thawa visthara danaganna oneda?

Describe the emergency... (Shift+Enter for new line)

Send

[illegible]

Smart First-Aid Assistant

AI-guided emergency first-aid, grounded in IFRC guidelines

Call 1990

Aria

READY

First-Aid & Health Guide

Gemini + RAG

EN · SI

31525es

Turn Around

Change Avatar

Clear Chat

Not a substitute for professional medical advice. In a real emergency, call 1990 immediately.

Aria 12:51 AM

පිලිස්සීම බරපතල බවටමක පවතී නම්, ප්‍රථමාධාර දීමට පෙර වහාම ****1990**** (සුවසිරිය) ඉමනා හදිසි ගිලුන්රථ සේවාව ලබාගන්න.

සාමාන්‍ය පිලිස්සුම් තුවාල සඳහා වන සම්මත ප්‍රථමාධාර පියවර පිළිබඳ සවිස්තර තොරතුරු මගේ ලේඛනවල ඉඩංගු නොවේ. කෙසේ වෙතත්, ලබා දී ඇති වෛද්‍ය මාර්ගෝපදේශවලට අනුව පහත කරුණු දැක්විය හැකිය:

- **කෙසෙල් කොළ වෙළුම් (Banana leaf dressings):**** සාමාන්‍ය ගෝන් වෙළුම්වලට වඩා කෙසෙල් කොළ වෙළුම් භාවිතයෙන් පිලිස්සුම් තුවාලවල වේදනාව, අපහසුතාවය සහ තුවාලය සුව වීමට ගතවන කාලය අඩු වන බව පර්යේෂණ මගින් පෙන්වා දී ඇත.
- **මේද අම්ල (Fatty acids):**** 'එනිල් ලීනොලේට්' (ethyl linoleate) වැනි මේද අම්ල ඉඩංගු ආලේපන මගින් වේදනා නාශක ඉවයන්තාවය සහ සුව වීමට ගතවන කාලය අඩු කරන බව සොයාගෙන ඇත.

ඔබට මේ පිළිබඳව විස්තර අවශ්‍යද?

මෙය වෘත්තීය වෛද්‍ය උපදෙසක් නොවන බව කරුණාවෙන් සලකන්න.

who-first-aid-guidelines.pdf

sl-emergency-numbers.txt

Copy

Describe the emergency... (Shift+Enter for new line)

Send

Create Your Avatar

Design your look, then click "Next" inside the editor when you're done.

Avaturn.

Customize your look and then click 'Next'

Next

Body

Skin tone correction

The screenshot shows the Visual Studio Code interface with the Explorer view on the left displaying the project structure of 'HEALTHCARE-FIRSTAID-ASSISTANT'. The main editor area shows the terminal output for running the server. The terminal text is as follows:

```
)  
[History restored]  
Microsoft Windows [Version 10.0.22000.8555]  
(c) Microsoft Corporation. All rights reserved.  
C:\Users\user\Desktop\healthcare-firstaid-assistant>cd server  
C:\Users\user\Desktop\healthcare-firstaid-assistant\healthcare-firstaid-assistant\server>npm run dev  
> server@1.0.0 dev  
> nodemon index.js  
  
[nodemon] 3.1.14  
[nodemon] to restart at any time, enter "rs"  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting "node index.js"  
O injected env (2) from .env // tip: # multiple files [ paths: ['.env.local', '.env'] ]  
Loaded 580 chunks from cache (documents unchanged) - no API calls needed.  
Server running on port 5000 - knowledge base ready  
[]
```

The screenshot shows the Visual Studio Code interface with the Explorer view on the left displaying the project structure of 'HEALTHCARE-FIRSTAID-ASSISTANT'. The main editor area shows the terminal output for running the client. The terminal text is as follows:

```
C:\Users\user\Desktop\healthcare-firstaid-assistant\healthcare-firstaid-assistant  
C:\Users\user\Desktop\healthcare-firstaid-assistant\healthcare-firstaid-assistant\client>npm run dev  
> client@0.0.0 dev  
> vite  
  
VITE v8.1.5 ready in 360 ms  
→ Local: http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help  
[]
```

Setup Instructions

- **Backend setup**

cd server

npm install

Copy .env.example to .env and add your GEMINI_API_KEY

npm run dev

(first run embeds the knowledge base and caches it to disk; later runs load instantly from cache)

- **Frontend setup**

cd client

npm install

npm run dev

Open the local URL shown in the terminal, create your avatar, and start chatting.

First run: what to expect

- The very first screen you see is the avatar creator (Avaturn), not the chat. This is intentional; the app checks whether an avatar has already been saved, and since it hasn't yet, it shows the creator first.
- Design your avatar inside the embedded creator, then click "Next"/"Done" to export it. The app receives the avatar's .glb URL automatically and switches to the main chat screen.
- The chosen avatar is then saved in the browser's localStorage, so closing or refreshing the page later goes straight to the chat screen instead of the creator again.
- From the chat screen, you can describe a first-aid scenario (in English, Sinhala, or Singlish) and the assistant will respond using the RAG-grounded knowledge base. The "Change Avatar" button in the sidebar returns to the creator at any time if you want to redesign it.

Assumptions

- The IFRC first-aid guideline PDF is present in server/knowledge/documents/ before the first run.
- A valid, non-rate-limited Gemini API key is available for the first (embedding) run; subsequent runs work even offline from Gemini's embedding API since results are cached.

7. Limitations & Future improvements

Limitations

- The avatar does not lip-sync to responses.
- Persistence is local to the browser (localStorage), not synced across devices.
- The embeddings cache is a flat JSON file, not a proper vector database. It is adequate for this knowledge-base size but would not scale to a much larger document set.

Future improvements

- Voice input/output for a hands-free emergency workflow.
- A proper vector database (e.g. Chroma, Pinecone) if the knowledge base grows significantly.
- Expand the first-aid knowledge base beyond the current IFRC guideline document.